山东大学 计算机科学与技术 学院

数据结构与算法 课程实验报告

学号：202200130077	姓名：薛润泽	班级：人工智能
实验题目：实验三 数组描述线性表		
实验学时：2	实验日期： 2023.9.27	

实验目的：

1.掌握线性表结构、数组描述方法（顺序存储结构）、数组描述线性表的实现。

2.掌握线性表应用。

软件开发工具：

DEV C++

1. 实验内容

设通讯录中每一个联系人的内容有：姓名、电话号码、班级、宿舍。由标准输入读入联系人信息，使用线性表中操作实现通讯录管理功能，包括：插入、删除、编辑、查找（按姓名查找）；键盘输入一班级，输出通讯录中该班级中所有人的信息。

2. 数据结构与算法描述 （整体思路描述，所需要的数据结构与算法）

思路：

构建类 Student 和类 AddressBook，在 Student 中封装一个学生的数据信息，包括 name，phonenumber，cls，dorm 等。并通过 Student 构造函数，将学生的数据信息 new 出。

在 AddressBook 中，封装一个 Student**类型的二维数组，用来指向 Student*。AddressBook 中封装的函数有:

(1)执行题目要求的插入记录操作 void addressinsert()

(2)执行删除操作 void addressdelete()

(3)执行根据姓名编辑一条记录操作 void addressupdate()

(4)执行根据姓名查找操作 int addressfind()其中该函数如果成功找到返回索引，找不到返回-1，再写一个函数用来将找到输出 1，没找到输出 0

(5)执行计算班级所有成员宿舍号的异或值操作 int addressdormcal()

(6)检查传入的两个姓名是否相同的函数 int studentnamecheck(char*A,char*B)

实现细节：

对于上述(1)因为在 Student 类中已经实现了构造函数 new 出一个学生的所有信息，并传入信息。在 void addressinsert()函数中，只需要再 people 数组中 new 一个 Student 类型的对象，并且 cnt++即可。

对于上述(2),addressfind()函数找到我们要删除的元素的索引,然后将 people 数组中该位置的 Student 对象释放，并将数组自删除元素开始整体向前移动一位。

对于上述(3),addressfind()函数找到我们要修改的元素的索引，然后判断我们的修改操作类型。并对相应元素进行修改即可。要注意的是，phonenumber 由一串字符串记录，而 cls 和 dorm 是 int 类型，使用时要对地址解引用。

对于上述(4)，从数组第一个元素开始判断名字是否与输入的名字相同，如果相同则输出索引，如果不相同则继续遍历数组，直到数组结束。其中用到了自定义函数 studentnamecheck,用来判断输入的两个字符串是否一致。

对于上述(5)，遍历数组，寻找班级号与要求班级相同的元素，并将 dorm 值异或到

tempans 中，最终返回 tempans。开始时，tempans 设置为 0。

对于上述(6),进行两个字符串的逐字符比较，如果两个字符串直到检查到’\0’都完全相同，则输出 1，否则输出 0。

3. 测试结果（测试输入，测试输出）



4. 分析与探讨（结果分析，若存在问题，探讨解决问题的途径）

结果与预期相同。注意二维数组的含义，合理正确运用二维数组。

5. 附录：实现源代码（本实验的全部源程序代码，程序风格清晰易理解，有充分的注释）

```
#include<iostream>

class Student{
public:
    Student(){
        name = new char[20];
        phonenumber = new char[20];
        cls = new int;
        dorm = new int ;
        std::cin>>name>>phonenumber>>*cls>>*dorm;
```

```
    }                   //每 new 一个 Student 对象，都会调用构造函数输入该 student 的
name, phonenumber class), drom
    ~Student(){
        delete []name;
        delete[]phonenumber;
        delete cls;
        delete dorm;  //对象析构要把所有 new 出的空间释放掉
    }
    char * name;          //姓名
    char * phonenumber;  //电话号码
    int * cls;            //存放班级
    int  *dorm;           //宿舍
};
class AddressBook{
public:
    AddressBook(int n){
        people = new Student*[n];   //因为有 n 个操作，所以在 n 个操作中最多插入 n
个 Student 对象
        cnt = 0;                  //将 cnt 初始为 0
    }
    ~AddressBook(){
        delete []people;          //删除指针数组
    }
    void addressinsert();
    int studentnamecheck(char* A, char* B);//根据姓名，检查传入的两个姓名是否相
等，如果相等则返回
    int addressfind();            //根据姓名查找 people 数组中与满足姓名的 student，
返回索引
    void addressdelete();
    void addressupdate();
    int addressdormcal();
private:
    Student** people;         //people 是一个 Student 类型二维数组
    int cnt;                  //cnt 记录 people 数组中实际元素个数
};

void AddressBook::addressinsert(){

    people[cnt] = new Student() ; //构造函数自动调用，同时写入姓名等信息
    cnt++;
}
int AddressBook::studentnamecheck(char *A, char* B){//  检查操作名字和对应学生
名字，两个名字是否相同，相同就返回 1，不同就返回 0
    int sym = 0;
    while(A[sym]==B[sym]){
        if(A[sym]=='\0'&&B[sym] =='\0') return 1;
```

```
        sym++;
    }
    return  0;
}
int AddressBook::addressfind(){                          // 根据姓名查找
    char* tempname = new char[20];
    std::cin>>tempname;                                  //输入要查找的姓名
    for(int i = 0; i < cnt; i++){
        if(studentnamecheck(tempname,people[i]->name)){ //从数组第一个元素开始
寻找，直到找到目标元素或者到最后一个元素
            delete[]tempname;                           //删除输入查找的名字
            return i;                                   //找到了则返回查找到的索引!!
索引
        }
    }
    delete []tempname;
    return -1;                                          //没找到，输出-1
}

void AddressBook::addressdelete(){
    int location = addressfind();                       //记录要被删除元素的索引
    if(location==-1) std::cout<<"Wrong name";
    else {
        delete people[location];
        for(int i = location; i < cnt - 1; i++){
            people[i] = people[i + 1];
            people[i + 1] =nullptr;
        }
    }
    cnt--;
}

void AddressBook::addressupdate(){
    int location = addressfind();                       //记录要被修改元素的索引
    int sym = 0;
    std::cin>>sym;                                      //记录更新操作类型
    if(sym == 1){
        std::cin>>people[location]->phonenumber;
    }
    else if(sym == 2){
        std::cin>>*people[location]->cls;
    }
    else if(sym == 3){
        std::cin>>*(people[location]->dorm);
    }
}
```

```
int AddressBook::addressdormcal(){                    //计算宿舍异或值
    int classnum;
    std::cin>>classnum;
    int tempans = 0;
    for(int i = 0; i < cnt; i++){                     //判断该学生是否满足班级
要求
        if(*people[i]->cls == classnum)
        tempans = tempans^*(people[i]->dorm);
    }
    return tempans;
}
int main(){
    int n;
    std::cin>>n;
    AddressBook addressbook(n);
    int opt;
    for(int i = 0; i < n; i++){
        std::cin>>opt;
        if(opt == 0){
            addressbook.addressinsert();
        }
        else if(opt == 1){
            addressbook.addressdelete();
        }
        else if(opt == 2){
            addressbook.addressupdate();
        }
        else if(opt == 3){
            int ans = addressbook.addressfind();
            if(ans == -1) std::cout<<'0'<<'\n';
            else std::cout<<'1'<<'\n';
        }
        else if(opt == 4){
            int ans = addressbook.addressdormcal();
            std::cout<<ans<<'\n';
        }
    }
    return 0;
}
```